

Assignment 1: Builder Pattern

Study report in plain text and HTML

Astana IT University | ShP-2216 - Software Design Patterns | Java 17

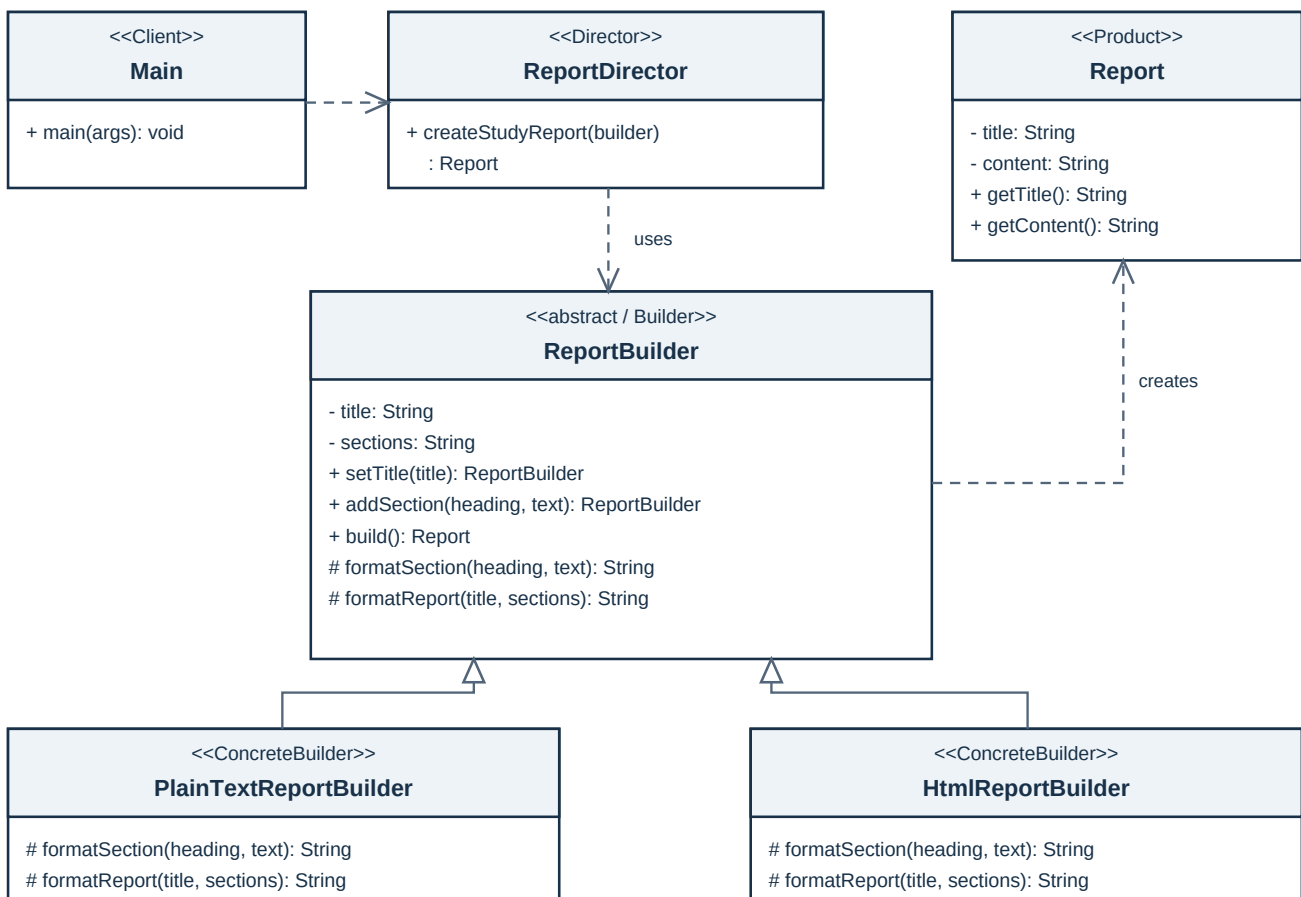
1. Introduction

The product is a study report with a title and several sections. Each section has a heading and some text. Builder lets the program add these parts step by step and create the finished Report by calling build().

There are two concrete builders. PlainTextReportBuilder uses line breaks, while HtmlReportBuilder uses HTML tags and a complete HTML document. Both receive the same steps from ReportDirector. This shows how one construction process can produce different representations.

Report is a normal class with a constructor, two private fields and getters. ReportBuilder is an abstract class with shared construction steps and two abstract formatting methods. Main chooses the builders and prints the results. The example uses only basic classes, inheritance, methods, strings and conditions.

2. UML class diagram



Hollow triangles show inheritance. Dashed arrows show use or creation. Main also creates both concrete builders and prints their reports; those extra dependencies are included in the editable docs/uml.puml source. The Builder class stores data, the concrete builders format it, and the Director chooses the sequence of steps.

3. Clean Code examples

1. Meaningful names

Names describe report-writing actions. Reading this chain shows what will be built without needing extra comments. Source: Main.java.

```
.setTitle("My homework")
.addSection("Task", "Practice method chaining")
.build();
```

2. Small methods

Each formatting method has one task. This method formats only one plain text section; formatReport() adds the report title. Source: PlainTextReportBuilder.java.

```
protected String formatSection(String heading, String text) {
    return heading + "\n" + text + "\n\n";
}
```

3. Avoid duplicated construction logic

Both builders inherit addSection(), so validation and adding a section are written once. The call to formatSection() uses the implementation of the selected concrete builder. Source: ReportBuilder.java.

```
checkText(heading, "Heading");
checkText(text, "Section text");
sections += formatSection(heading, text);
return this;
```

4. Validate the result

build() gives a clear error when required parts are missing. There is a similar check for missing sections. Source: ReportBuilder.java.

```
if (title.isBlank()) {
    throw new IllegalStateException(
        "Add a title before building the report");
}
```

5. Keep fields private

Other classes read a finished report through getters. They cannot directly replace its fields. final means these fields are assigned in the constructor and cannot be reassigned. Source: Report.java.

```
private final String title;
private final String content;

public String getContent() {
    return content;
}
```

4. Running and checking the program

Run Main.java in IntelliJ IDEA with JDK 17, or run the commands below from the repository folder. The first command prints a plain text report, an HTML report and a custom report built without a Director. No files or PDFs are generated by the Java program.

```
bash scripts/run.sh
bash scripts/test.sh
```

The test class has five small methods. They check the plain text result, HTML output and escaped text, chaining with return this, a missing title, and missing sections. All five passed on Amazon Corretto 17.0.13. GitHub Actions is configured to run the same checks on Java 17.

A typical call with a Director is shown below. Replacing PlainTextReportBuilder with HtmlReportBuilder changes the output representation while the Director's steps stay the same.

```
ReportDirector director = new ReportDirector();
Report report = director.createStudyReport(
    new PlainTextReportBuilder());
System.out.println(report.getContent());
```

5. Conclusion

Builder makes the construction steps easy to read. A client can add sections one by one, and the same Director can work with two different builders. Keeping common steps in ReportBuilder avoids repeating validation in both concrete classes.

One tradeoff in this implementation is choosing where to put shared code. A separate interface plus an extra base class added more types to explain. Using one abstract Builder keeps the example smaller while still sharing construction logic. Another tradeoff is using simple string concatenation: it is easy to understand for short reports, but large documents would benefit from a different approach.

The pattern also adds more classes than a constructor alone. A constructor would be enough for a small object with a fixed number of fields. This example uses Builder because sections can be added gradually and two representations are required. Each new report uses a fresh builder because build() does not reset stored data.

6. Repository

GitHub: [unknown9error/aitu-software-design-pattern-assignment1](https://github.com/unknown9error/aitu-software-design-pattern-assignment1)

The repository contains source code, a README, incremental commits, example output, UML and this report. The implementation follows the supplied Assignment 1 requirements. This PDF is a separate written report, not a feature of the Java program.